

Deep Learning Final Project

DENOISING AUTOENCODERS

DL Code - 1 :

```
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np

class Encoder(nn.Module):
    def __init__(self):
        super(Encoder, self).__init__()
        self.cnn1 = nn.Conv2d(1, 64, 5, stride=2, padding=2)
        self.cnn2 = nn.Conv2d(64, 128, 5, stride=2, padding=2)

    def forward(self, x):
        x = F.relu(self.cnn1(x))
        return self.cnn2(x)

class Decoder(nn.Module):
    def __init__(self):
        super(Decoder, self).__init__()
        self.conv1 = nn.ConvTranspose2d(128, 64, 5, stride=1, padding=2,
        bias=False) self.conv2 = nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1,
        bias=False) self.conv3 = nn.ConvTranspose2d(32, 1, 4, stride=2, padding=1,
        bias=False)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        return self.conv3(x)

encoder = Encoder()
decoder = Decoder()

optimizer = optim.Adam(encoder.parameters())
```

```
optim.Adam(encoder.parameters()) optim2  
= optim.Adam(decoder.parameters())  
loss_fn = nn.MSELoss()
```

```
batch_size_train = 256  
n_epochs = 1
```

```
train_loader = torch.utils.data.DataLoader(
```

```

torchvision.datasets.MNIST('/data/', train=True,
    download=True,
    transform=torchvision.transforms.ToTensor()),
batch_size=batch_size_train, shuffle=True)

for epoch in range(n_epochs):
    for batch_idx, (data, target) in
        enumerate(train_loader): x = encoder(data)
        y = decoder(x)
        train_loss = loss_fn(y,
            data) if batch_idx % 10 ==
            0:
                print('train', epoch, batch_idx, float(train_loss))
            optim1.zero_grad()
            optim2.zero_grad()
            train_loss.backward()
            optim1.step()
            optim2.step()

torch.save(encoder.state_dict(), "DLApr8Aencoderstate.pt")
torch.save(decoder.state_dict(), "DLApr8Adecoderstate.pt")

```

OUTPUT :

```

train 0 0 0.11441943049430847
train 0 10 0.05015511438250542
train 0 20 0.02593315951526165
train 0 30 0.015622702427208424
train 0 40 0.011079028248786926
train 0 50 0.008188702166080475
train 0 60 0.006732837297022343
train 0 70 0.005456315819174051
train 0 80 0.005413891281932592
train 0 90 0.004715980030596256
train 0 100 0.003991877660155296
train 0 110 0.0036293398588895798
train 0 120 0.0034257543738931417
train 0 130 0.002977213356643915
train 0 140 0.0028539129998534918
train 0 150 0.0024748833384364843
train 0 160 0.002290536416694522
train 0 170 0.002180092968046665
train 0 180 0.002014787634834647
train 0 190 0.0019194617634639144
train 0 200 0.0017931615002453327
train 0 210 0.0016113893361762166
train 0 220 0.0029577233362942934
train 0 230 0.0020030755549669266

```

DL Code- 2 :

```
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

testset = torchvision.datasets.MNIST('/data/', train=False,
                                     transform=torchvision.transforms.ToTensor())
images = torch.tensor([x[0].clone().detach().numpy() for x in testset])
test_size = len(images)

class Encoder(nn.Module):
    def __init__(self):
        super(Encoder, self).__init__()
        self.cnn1 = nn.Conv2d(1, 64, 5, stride=2, padding=2)
        self.cnn2 = nn.Conv2d(64, 128, 5, stride=2, padding=2)

    def forward(self, x):
        x = F.relu(self.cnn1(x))
        return self.cnn2(x)

class Decoder(nn.Module):
    def __init__(self):
        super(Decoder, self).__init__()
        self.conv1 = nn.ConvTranspose2d(128, 64, 5, stride=1, padding=2,
                                         bias=False)
        self.conv2 = nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1,
                                         bias=False)
        self.conv3 = nn.ConvTranspose2d(32, 1, 4, stride=2, padding=1,
                                         bias=False)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        return self.conv3(x)

encoder = Encoder()
decoder = Decoder()
encoder.load_state_dict(torch.load("DLApr8Aencoderstate.pt"))
decoder.load_state_dict(torch.load("DLApr8Adecoderstate.pt"))

x = encoder(images)
```

```
mean = torch.mean(x,  
0)  
x = mean.view((1, 128, 7, 7))  
y = decoder(x)
```

```

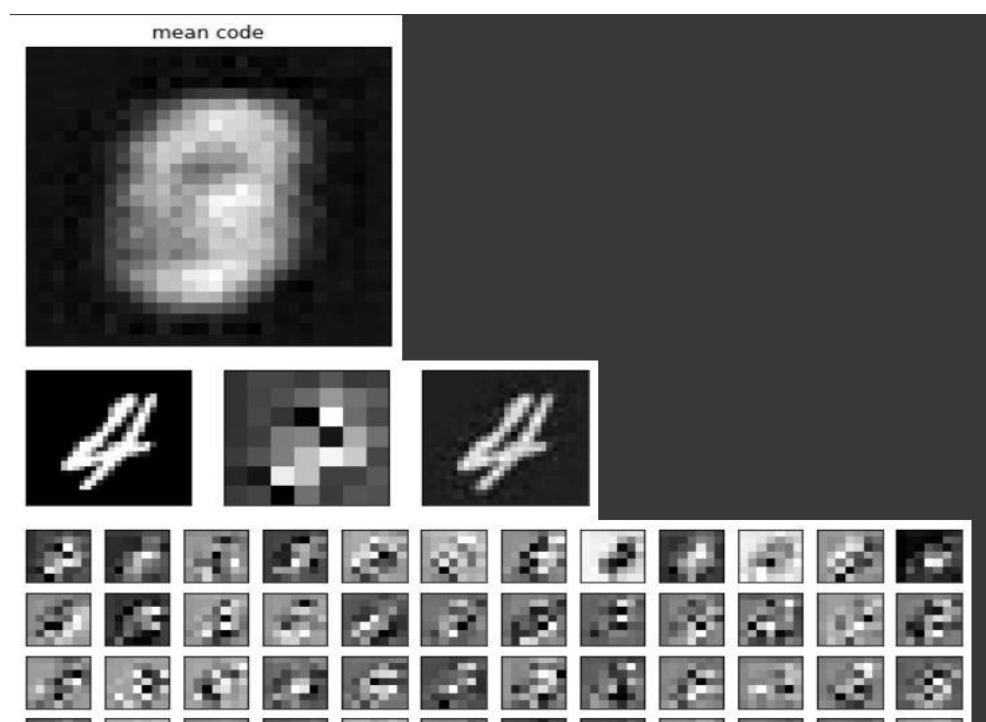
plt.imshow(y[0][0].detach().numpy(), cmap='gray')
plt.xticks([])
plt.yticks([])
plt.title('mean code')
plt.show()

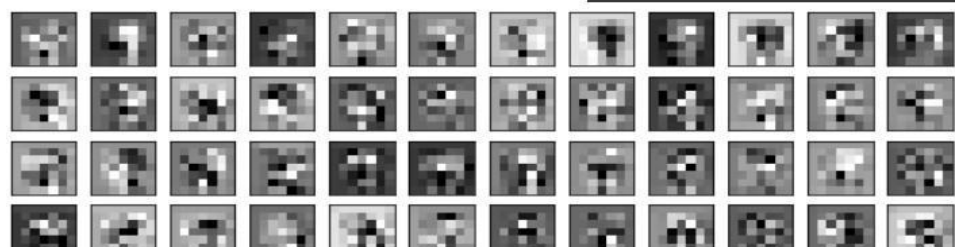
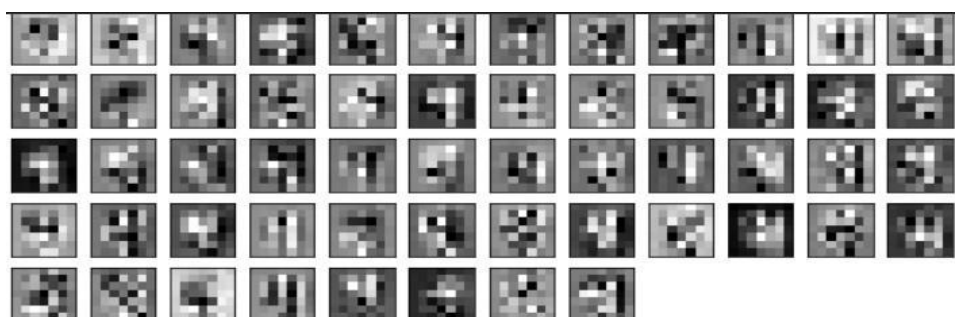
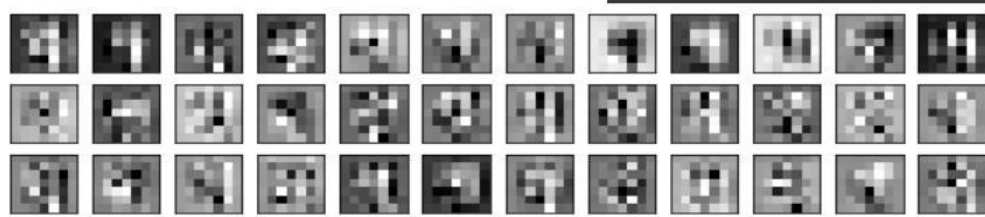
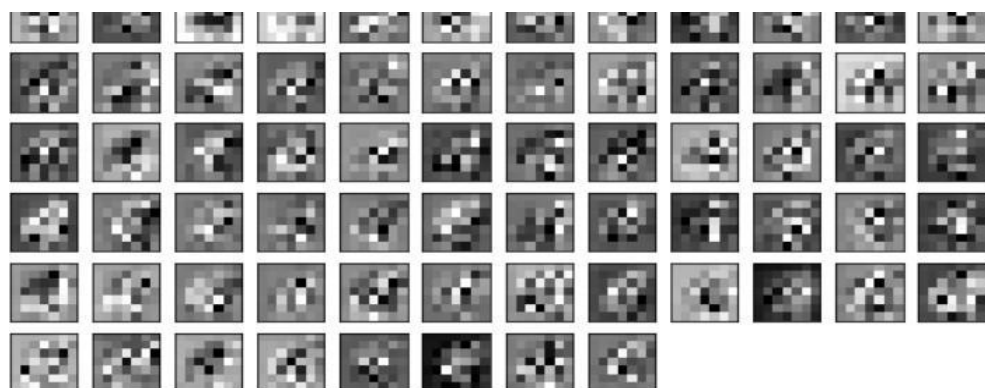
for i in range(5):
    index = np.random.randint(test_size)
    sample, t1 = testset[index]
    s = sample.view((1, 1, 28, 28))
    x = encoder(s)
    y = decoder(x)
    plt.subplot(1, 3,
    1)
    plt.imshow(s[0][0], cmap='gray')
    plt.xticks([])
    plt.yticks([])
    plt.subplot(1, 3,
    2)
    plt.imshow(x[0][0].detach().numpy(), cmap='gray')
    plt.xticks([])
    plt.yticks([])
    plt.subplot(1, 3,
    3)
    plt.imshow(y[0][0].detach().numpy(), cmap='gray')
    plt.xticks([])
    plt.yticks([])
    plt.show()

plt.figure(figsize=(10,
10)) for j in range(128):
    plt.subplot(12, 12, j + 1)
    plt.imshow(x[0][j].detach().numpy(), cmap='gray')
    plt.xticks([])
    plt.yticks([])
    plt.show()

```

OUTPUT :







```

import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

testset = torchvision.datasets.MNIST('/data/', train=False,
                                     transform=torchvision.transforms.ToTensor())
test_size = len(testset)

class Encoder(nn.Module):
    def __init__(self):
        super(Encoder, self).__init__()
        self.cnn1 = nn.Conv2d(1, 64, 5, stride=2, padding=2)
        self.cnn2 = nn.Conv2d(64, 128, 5, stride=2, padding=2)

    def forward(self, x):
        x = F.relu(self.cnn1(x))
        return self.cnn2(x)

class Decoder(nn.Module):
    def __init__(self):
        super(Decoder, self).__init__()
        self.conv1 = nn.ConvTranspose2d(128, 64, 5, stride=1, padding=2,
                                         bias=False)
        self.conv2 = nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1,
                                         bias=False)
        self.conv3 = nn.ConvTranspose2d(32, 1, 4, stride=2, padding=1,
                                         bias=False)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        return self.conv3(x)

encoder = Encoder()
decoder = Decoder()
encoder.load_state_dict(torch.load("DLApr8Aencoderstate.pt"))
decoder.load_state_dict(torch.load("DLApr8Adecoderstate.pt"))

for i in range(5):
    index = np.random.randint(test_size)
    sample, t1 = testset[index]
    s = sample.view((1, 1, 28, 28))
    s2 = s.clone()

```

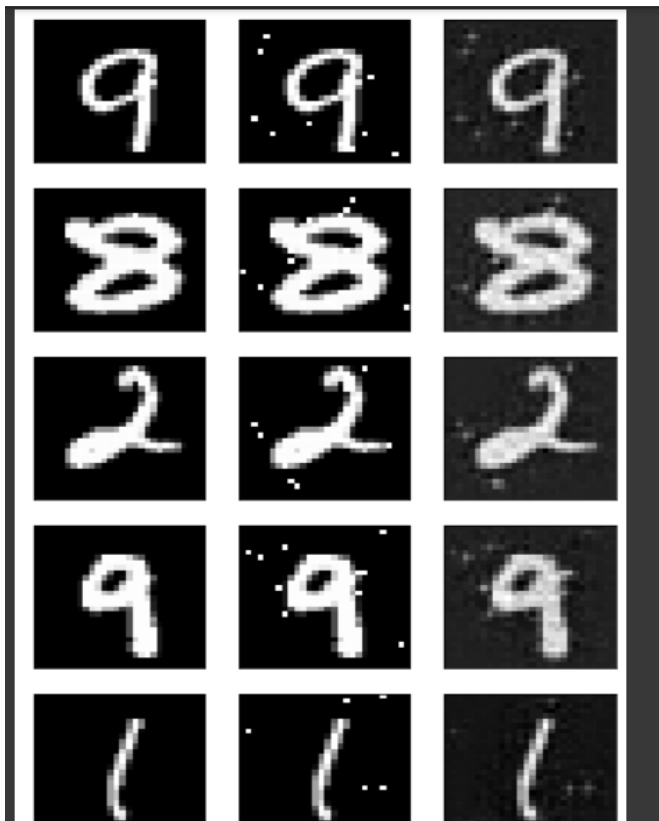
```
for j in range(10):  
    row = np.random.randint(28)  
    col = np.random.randint(28)
```

```

s2[0][0][row][col] = 1
x = encoder(s2)
y = decoder(x)
plt.subplot(1, 3,
1)
plt.imshow(s[0][0], cmap='gray')
plt.xticks([])
plt.yticks([])
plt.subplot(1, 3,
2)
plt.imshow(s2[0][0].detach().numpy(), cmap='gray')
plt.xticks([])
plt.yticks([])
plt.subplot(1, 3,
3)
plt.imshow(y[0][0].detach().numpy(), cmap='gray')
plt.xticks([])
plt.yticks([])
plt.show()

```

OUTPUT :



DL Code 4:

```

import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np

```

```

class Encoder(nn.Module):
    def __init__(self):
        super(Encoder, self).__init__()
        self.cnn1 = nn.Conv2d(1, 64, 5, stride=2, padding=2)
        self.cnn2 = nn.Conv2d(64, 128, 5, stride=2, padding=2)

    def forward(self, x):
        x = F.relu(self.cnn1(x))
        return self.cnn2(x)

```

```

class Decoder(nn.Module):
    def __init__(self):
        super(Decoder, self).__init__()
        self.conv1 = nn.ConvTranspose2d(128, 64, 5, stride=1, padding=2,
        bias=False) self.conv2 = nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1,
        bias=False) self.conv3 = nn.ConvTranspose2d(32, 1, 4, stride=2, padding=1,
        bias=False)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        return self.conv3(x)

```

```

encoder = Encoder()
decoder =
Decoder()

```

```

optim1 =
optim.Adam(encoder.parameters()) optim2
= optim.Adam(decoder.parameters())
loss_fn = nn.MSELoss()

```

```

batch_size_train = 256
n_epochs = 1

```

```

train_loader = torch.utils.data.DataLoader(
    torchvision.datasets.MNIST('/data/', train=True,
    download=True,

```

```
    transform=torchvision.transforms.ToTensor()),  
    batch_size=batch_size_train, shuffle=True)  
  
for epoch in range(n_epochs):  
    for batch_idx, (data, target) in enumerate(train_loader):
```

```

batchsize = len(data)
data2 = data.clone()
for i in range(batchsize):
    for j in range(10):
        row = np.random.randint(28)
        col = np.random.randint(28)
        data2[i][0][row][col] = 1
x = encoder(data2)
y = decoder(x)
train_loss = loss_fn(y,
data) if batch_idx % 10 ==
0:
    print('train', epoch, batch_idx, float(train_loss))
optim1.zero_grad()
optim2.zero_grad()
train_loss.backward()
optim1.step()
optim2.step()

torch.save(encoder.state_dict(), "DLApr8Dencoderstate.pt")
torch.save(decoder.state_dict(), "DLApr8Ddecoderstate.pt")

```

OUTPUT :

```

train 0 0 0.11358359456062317
train 0 10 0.0557817667722702
train 0 20 0.029220594093203545
train 0 30 0.019033627584576607
train 0 40 0.014548714272677898
train 0 50 0.011670100502669811
train 0 60 0.012263242155313492
train 0 70 0.008962876163423061
train 0 80 0.007790634408593178
train 0 90 0.006999006029218435
train 0 100 0.006547224707901478
train 0 110 0.0060728248208761215
train 0 120 0.0056805238127708435
train 0 130 0.005623016506433487
train 0 140 0.005114397965371609
train 0 150 0.004769835155457258
train 0 160 0.004605137277394533
train 0 170 0.004738750401884317
train 0 180 0.004226449877023697
train 0 190 0.003927149344235659
train 0 200 0.0039331610314548016
train 0 210 0.003691910533234477
train 0 220 0.0034389495849609375
train 0 230 0.003576357150450349

```

DL Code- 3 Modified Code :

```
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

testset = torchvision.datasets.MNIST('/data/', train=False,
                                     transform=torchvision.transforms.ToTensor())
test_size = len(testset)

class Encoder(nn.Module):
    def __init__(self):
        super(Encoder, self).__init__()
        self.cnn1 = nn.Conv2d(1, 64, 5, stride=2, padding=2)
        self.cnn2 = nn.Conv2d(64, 128, 5, stride=2, padding=2)

    def forward(self, x):
        x = F.relu(self.cnn1(x))
        return self.cnn2(x)

class Decoder(nn.Module):
    def __init__(self):
        super(Decoder, self).__init__()
        self.conv1 = nn.ConvTranspose2d(128, 64, 5, stride=1, padding=2,
                                         bias=False)
        self.conv2 = nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1,
                                         bias=False)
        self.conv3 = nn.ConvTranspose2d(32, 1, 4, stride=2, padding=1,
                                         bias=False)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        return self.conv3(x)

encoder = Encoder()
decoder = Decoder()
encoder.load_state_dict(torch.load("DLApr8Dencoderstate.pt"))
decoder.load_state_dict(torch.load("DLApr8Ddecoderstate.pt"))

for i in range(5):
    index = np.random.randint(test_size)
```

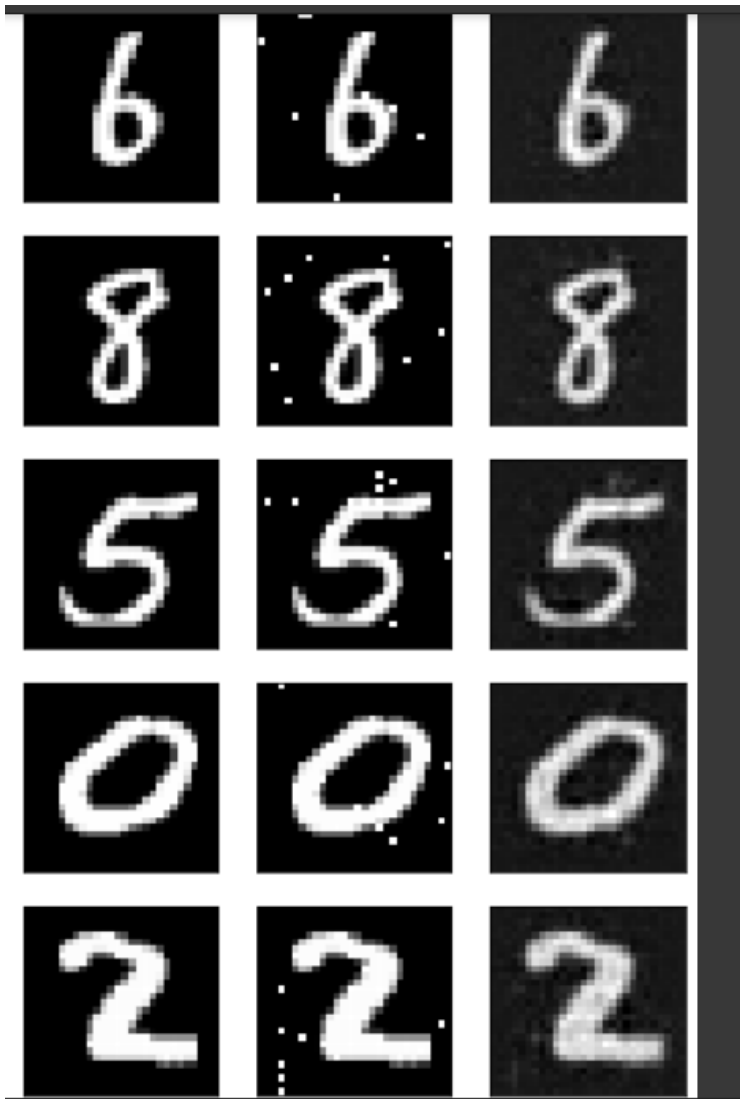
```
sample, t1 = testset[index]  
s = sample.view((1, 1, 28, 28))  
s2 = s.clone()
```

```

for j in range(10):
    row = np.random.randint(28)
    col = np.random.randint(28)
    s2[0][0][row][col] = 1
x = encoder(s2)
y = decoder(x)
plt.subplot(1, 3, 1)
plt.imshow(s[0][0], cmap='gray')
plt.xticks([])
plt.yticks([])
plt.subplot(1, 3,
2)
plt.imshow(s2[0][0].detach().numpy(), cmap='gray')
plt.xticks([])
plt.yticks([])
plt.subplot(1, 3,
3)
plt.imshow(y[0][0].detach().numpy(), cmap='gray')
plt.xticks([])
plt.yticks([])
plt.show()

```

OUTPUT:



Comments :

DL Code-3 Modified Code 's output has less noise compared to its original Code.

